



Optimization of Traffic Light Control Systems for Urban Intersections in Palapye

Full Name: Tsotlhe Nayang Seiphepi

Student ID: 21001137

Supervisor: Dr. Bokamoso Basutli

Supervisor's Signature:

Student's Signature:

Department of Computer and Telecommunications Engineering
Faculty of Engineering and Technology
Botswana International University of Science and Technology
Palapye, Botswana

In partial fulfillment of the requirements for the degree of

Bachelor of Computer and Telecommunications Engineering

2025

Contents

1	Introduction	1
1.1	Background of the study	1
1.2	Problem Statement	3
1.3	Research Objectives	4
2	Literature Review	5
2.1	Sensory Circuits	5
2.2	Processing and Control Logic	9
2.2.1	Fixed-Time Control Method	9
2.2.2	Actuated Control Method	9
2.2.3	Adaptive Control Method	10
2.3	Model-Based Optimization Systems	10
2.3.1	Model Predictive Control	10
2.3.2	Dynamic Traffic Assignment (DTA)	11
2.3.3	Mixed-Integer Linear Programming (MILP)	12
2.4	Model- free Optimization Systems	13
2.4.1	Reinforcement Learning	14
2.4.2	Reinforcement Learning (RL) Update Rule	14
3	Methodology	20

3.1	3.1 Overview of the Methodology	20
3.2	3.2 System Architecture and Dashboard Design	21
3.2.1	3.2.1 Dataset Training	21
4	Results and Analysis	30
4.1	4.1 Simulation Setup	30
4.2	4.2 Visual Output of the Simulations	30
4.2.1	4.2.1 Fixed-Time Traffic Signal Control	31
4.2.2	4.2.2 PPO Adaptive RL Controller	32
4.2.3	4.2.3 CameraVision / YOLO Detection (Optional)	33
4.3	4.3 Performance Metric Comparison	34
4.3.1	4.3.1 Average Delay	36
4.3.2	4.3.2 Queue Length	37
4.3.3	4.3.3 Throughput	37
4.3.4	4.3.4 Stop Ratio	38
4.3.5	4.3.5 Pressure	38
4.4	4.4 Discussion of Results	39
4.4.1	Overall Performance Summary	39
4.4.2	Delay	39
4.4.3	Queue Length	40
4.4.4	Throughput	41
4.4.5	Stop Ratio	41
4.4.6	Traffic Pressure	42
5	Discussion	43

5.0.1	Conclusion	43
6	Recommendations	45
7	Appendix A	48
7.1	PPO Training Script	48

List of Tables

4.1	Comparative Performance of PPO and Fixed-Time Controllers Across All Metrics	39
4.2	Phase-Specific Delay Performance ($\mu \pm \sigma$)	40
4.3	Phase-Specific Queue Length Performance ($\mu \pm \sigma$)	40
4.4	Phase-Specific Throughput Performance ($\mu \pm \sigma$)	41
4.5	Phase-Specific Stop Ratio Performance ($\mu \pm \sigma$)	41
4.6	Phase-Specific Traffic Pressure Performance ($\mu \pm \sigma$)	42

List of Figures

1.1	Traffic on the A1 road near Palapye (from Mmegi Online) [1]	2
2.1	Block Diagram of The System	5
2.2	Block diagram of the vehicle detection and counting system.	6
2.3	Block diagram of the LiDAR-based vehicle detection system.	8
2.4	Block Diagram of Reinforcement Learning System	14
2.5	High-level RL control loop for adaptive traffic signal control.	16
2.6	DQN architecture for traffic signal decision-making.	17
2.7	Proximal Policy Optimization (PPO) workflow.	18
2.8	Centralized training and decentralized execution in Multi-Agent RL.	19
3.1	Flow Diagram for Camera-Based Traffic Light Control with PPO and Decision Loop	22
3.2	Custom SUMO intersection used for dataset generation and DRL training.	24
3.3	YOLOv8 vehicle detection test on sample traffic footage.	25
4.1	SUMO running with the fixed-time traffic signal controller.	31
4.2	SUMO gannt chart showing phase durations with the fixed-time traffic signal controller.	32
4.3	SUMO running with the PPO adaptive RL traffic signal controller.	33
4.4	YOLO-based traffic detection module during testing.	34

4.5	PERFORMANCE METRICS FOR FIXED TRAFFIC LIGHT CONTROL SYSTEM	35
4.6	PERFORMANCE METRICS FOR PPO MODEL TRAFFIC LIGHT CONTROLLED SYSTEM	36
4.7	Average delay comparison between fixed-time and PPO controller.	36
4.8	Average queue length comparison between fixed-time and PPO controller. . .	37
4.9	Vehicle throughput comparison.	37
4.10	Stop ratio comparison.	38
4.11	Intersection pressure comparison.	38
7.1	Training the PPO model	51

Chapter 1

Introduction

Palapye is rapidly emerging as both a commercial hub and a strategic transit town, yet its current traffic management system has not kept pace with this growth. The reliance on static, pre-timed traffic signals has become increasingly inadequate in the face of rising vehicular volumes. During peak hours, public holidays, and special events, these systems fail to respond to changing traffic patterns, resulting in chronic congestion, excessive delays, and frustration among commuters.

The consequences go far beyond inconvenience. Prolonged idling leads to higher fuel consumption and air pollution, directly harming the environment and economy. Congestion at busy intersections also increases the likelihood of road accidents, putting public safety at risk. In turn, long and unpredictable waiting times reduce the quality of life for residents and diminish Palapye's attractiveness as a growing urban centre.

These challenges create a strong case for adopting a smarter solution. An intelligent, adaptive traffic control system, capable of adjusting signal timing in real time, offers a path toward safer, cleaner, and more efficient mobility. By reducing delays, improving connectivity, and enhancing overall commuter experience, such a system would not only solve today's pressing traffic issues but also future-proof Palapye's transport network as the town continues to expand.

1.1 Background of the study

Palapye has evolved from a village into a growing town, which is now a very strategically placed settlement. It is situated between Gaborone and Francistown, the two major cities of the country Botswana, connected by the A1 highway. This positioning also makes Palapye a natural gateway for traffic moving between South Africa and Zimbabwe [2]. Major assets such as the Morupule Coal Mine and the Botswana International University of Science and

Technology (BIUST) add recurrent, time-peaked demand from shifts, deliveries, and campus activity.

Road capacity and traffic control have not expanded at the same growth rate as the town's activity. Many signalised intersections still run on static, pre-timed cycles that do not respond to fluctuating flows. During peak travel times, public holidays, university openings, and special events, queues build on the A1 road and town access roads, causing longer delays and traffic congestion, higher fuel use, and elevated local gas emissions. Congestion at busy junctions increases crash risk and contributes to commuter frustration, as well as high speeding on the A1 roads as commuters try to reach their delayed destinations earlier. This context motivates a focused study of Palapye's traffic conditions and management options, establishing a baseline against which modern data-driven control strategies can be assessed.



Figure 1.1: Traffic on the A1 road near Palapye (from Mmegi Online) [1]

1.2 Problem Statement

Urban intersections in Palapye experience increased traffic congestion, delays, and inefficient traffic flow due to conventional fixed-time traffic light control systems that do not adapt to real-time traffic conditions .

1.3 Research Objectives

- Design and implement sensory systems that accurately detect vehicles, pedestrians, and traffic conditions in real time with high reliability and minimal false detection.
- Design and implement a networking framework that ensures seamless, low-latency, and secure transmission of traffic data between sensors, local controllers, and the processing unit, supporting both wired and wireless communication protocols.
- Design and implement intelligent algorithms that analyse incoming data, optimise signal timing dynamically, and integrate historical traffic patterns to minimise congestion, delay, and emissions while ensuring safety.
- Design and implement reliable output control systems that translate optimization decisions into precise operation of traffic signals, pedestrian crossings, and variable message signs in real time.

Chapter 2

Literature Review

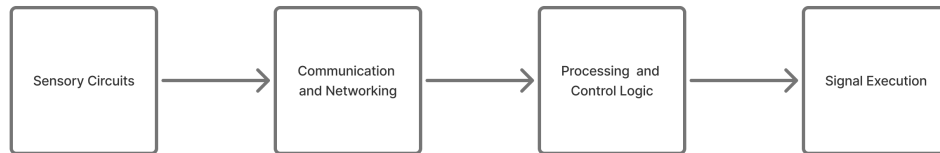


Figure 2.1: Block Diagram of The System

2.1 Sensory Circuits

Vehicle Detection using Camera Vision

The vehicle detection process in the paper by [3], is based entirely on classical computer vision techniques implemented using OpenCV. The system begins with video input, where frames are captured from a static camera. Each frame then undergoes background subtraction using the MOG2 algorithm to separate moving objects (vehicles) from the static background. After isolating the moving areas, the image is thresholded and binarized to create a black-and-white mask that clearly distinguishes motion from non-motion regions. Next, the system applies contour detection to identify the outlines of these moving objects. To remove noise and irrelevant small objects, contours below a specific area threshold (e.g., 500–1000 pixels) are discarded, keeping only the larger regions that likely represent vehicles. For each valid contour, the algorithm computes its centroid (center point) and monitors its position relative to a predefined reference line on the frame. When a centroid crosses this line within a set tolerance range, the system records it as a vehicle count event. This crossing logic ensures each vehicle is counted once as it passes through the scene. The final step

updates the vehicle count, allowing the program to estimate traffic flow. According to [3], this method achieved high accuracy for smaller vehicles (around 90 percent for cars and 100 percent for motorcycles) but lower accuracy for large vehicles such as buses or trucks due to occlusion and size variability.

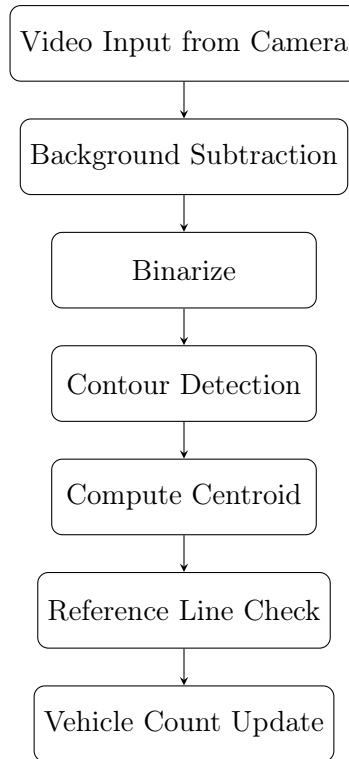


Figure 2.2: Block diagram of the vehicle detection and counting system.

Vehicle Detection Using LiDAR

In the paper “LiDAR-Enhanced Connected Infrastructures: Sensing and Broadcasting High-Resolution Traffic Information Serving Smart Cities” [4], the authors rely entirely on LiDAR sensing to detect, classify, and track vehicles and pedestrians in real time. The LiDAR sensor used—Velodyne VLP-16—emits multiple laser beams that rotate 360°, producing up to 600,000 distance points per second across a range of roughly 100 meters. Each reflected pulse provides a precise 3D coordinate (x, y, z), allowing the system to reconstruct the spatial structure of the road environment. When installed at the roadside about 7–9 feet high, the sensor captures complete 3D profiles of all moving road users within its field of view, independent of light or weather conditions. [4].

The LiDAR system detects vehicles by continuously scanning the surroundings with rotating laser beams that produce a 3D point cloud. Each laser pulse measures the distance to nearby surfaces, so cars appear as dense clusters of reflected points above the ground.

The process starts with background filtering, where stationary objects like roads and buildings are removed using multiple frames to isolate only moving points. Then, DBSCAN clustering groups those remaining points into individual objects based on their spatial density—each cluster likely representing a car, pedestrian, or cyclist. [4] For every cluster, the system extracts geometric features such as point count, object width, height, and distance from the sensor. These are passed into a Random Forest classifier trained to recognize the difference between vehicles and pedestrians. Vehicles show broader, lower, and denser point distributions than upright, narrow pedestrian clusters.

Once classified, each vehicle’s outline is approximated using a minimum-area rectangle, and its motion is tracked frame by frame with Kalman filtering and Global Nearest Neighbor association. This produces a continuous, accurate trajectory for every vehicle within the LiDAR’s 100-meter range. [4]

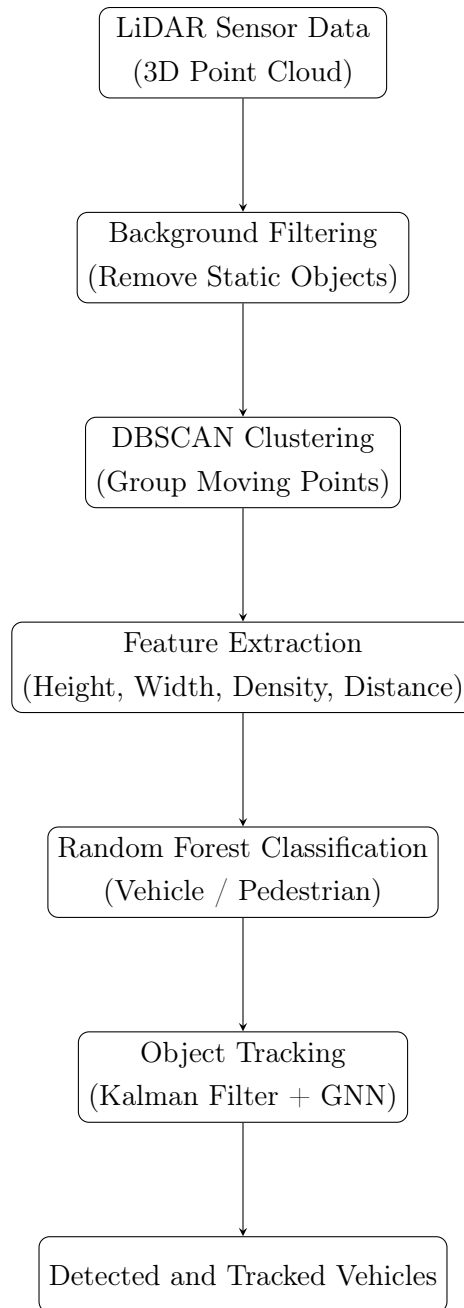


Figure 2.3: Block diagram of the LiDAR-based vehicle detection system.

Vehicle Detection Using Radio Waves and Microwaves

2.2 Processing and Control Logic

2.2.1 Fixed-Time Control Method

This technique relies on predetermined schedules for signal phases, where the green, yellow, and red times are fixed in advance. It has been shown to be effective in environments with stable and predictable traffic flows due to its simplicity and ease of implementation. However, the method does not adapt to real-time fluctuations in traffic demand. As a result, its performance deteriorates under conditions of variable density, such as during accidents, special events, or sudden surges in demand [5]. There are three common types of fixed-time control methods [6]:

1. **Isolated Fixed-Time Control:** Each intersection operates on its own predetermined schedule, with no coordination between adjacent intersections.
2. **Coordinated Fixed-Time Control:** Intersections are synchronized to create a green wave progression along major corridors, improving traffic flow over longer stretches.
3. **Time-of-Day Control:** Signal schedules vary by predefined time periods (e.g., morning peak, off-peak, evening rush, but remain non-responsive to live traffic conditions.

2.2.2 Actuated Control Method

The actuated control method adjusts signal timings dynamically based on real-time inputs from sensors such as loop detectors, cameras, and infrared devices [7]. Unlike fixed-time systems, which rely on predetermined schedules, actuated control responds directly to actual vehicle arrivals by extending green phases when vehicles are detected within the extension window. This responsiveness improves intersection efficiency by reducing unnecessary idling and abrupt stops.

Kalašová [7] investigated the impact of actuated signal control on both traffic flow and environmental outcomes. Their evaluation focused on key performance indicators, including average delay, queue length, fuel consumption, and emissions. The average delay per vehicle was defined as:

$$d = \frac{\sum(t_{\text{actual}} - t_{\text{free}})}{N} \quad (2.1)$$

where t_{actual} is the actual travel time

t_{free} is the free-flow travel time

N is the number of vehicles observed.

Queue length was measured as the accumulation of vehicles at red signals, while environmental indicators such as fuel consumption and emissions were modeled as functions of idling time and acceleration cycles.

The study showed that actuated control, which adjusts traffic signals based on real-time traffic, significantly cuts down on delays and queue lengths compared to fixed-time signals, especially in variable or moderate traffic conditions. It also reduces idling and stop-and-go driving, leading to lower CO₂ emissions and fuel use. However, in heavily congested, over saturated conditions, actuated control alone isn't enough to fully resolve congestion or delays.

Overall, research highlights actuated signal control as a substantial improvement over fixed-time methods, especially in urban environments where traffic demand fluctuates throughout the day [7].

2.2.3 Adaptive Control Method

The adaptive control method represent the latest and most advanced form of traffic signal management. They continuously monitor traffic density in real time and apply algorithmic models to optimize signals across networks. [8]. The adaptive models reduce traffic delays and adapt to real time traffic densities. The adaptive models are divided into two types. [9]

2.3 Model-Based Optimization Systems

These systems use mathematical models to represent how traffic behaves. Instead of trial and error, they predict what traffic will look and then decide the best traffic signal timings. Common approaches include:

2.3.1 Model Predictive Control

This technique predicts future traffic states like queue lengths and delays over a finite prediction horizon and computes the best signal timings in advance. A mathematical model predicts how traffic will evolve if certain green/red timings are applied. An optimization problem is solved to minimize a cost function subject to constraints like minimum green time or safety rules. The first part of the optimized signal plan is implemented. The process repeats at the next step with updated traffic data. [10]. Mathematically, the MPC problem

is expressed as:

$$\min_{u(0), \dots, u(N-1)} \sum_{k=0}^{N-1} J(x(k), u(k)) + J_f(x(N)) \quad (2.2)$$

subject to:

$$x(k+1) = f(x(k), u(k)) + d(k), \quad x(k) \in X, u(k) \in U \quad (2.3)$$

where

- $x(k)$ represents the traffic state (e.g., queue lengths or densities)
- $u(k)$ the control action (signal timings)
- $d(k)$ the demand disturbance.

The stage cost $J(x, u)$ typically combines queue lengths, waiting times, or number of stops, while $J_f(x(N))$ serves as a terminal cost penalizing undesirable end states.

[10] found out that MPC can anticipate future conditions, explicitly handle multi-variable intersections and constraints. Achieves better performance compared to fixed and actuated control strategies. Distributed and hierarchical MPC variants also offer improved coordination among multiple intersections.

However [10] highlighted critical limitations of the MCP. Centralized MPC is poor at computational complexity that restricts real time feasibility in large urban networks, performance is highly dependent on the accuracy of traffic models, and most applications remain at the simulation stage, with few large-scale real-world deployments. Scalability and robustness to uncertainty remain open challenges. [10]

2.3.2 Dynamic Traffic Assignment (DTA)

Dynamic Traffic Assignment (DTA) extends static traffic assignment by considering the time-varying nature of traffic demand and network conditions. The goal is to allocate vehicles dynamically across available routes to minimize congestion and balance traffic loads. Unlike static methods, which assume fixed flows, DTA accounts for fluctuations in demand, travel times, and congestion propagation throughout the network. This makes it suitable for analyzing real-world urban environments where traffic conditions evolve continuously. [11]

Dynamic Traffic Assignment (DTA) is often formulated as a Dynamic User Equilibrium (DUE), where no driver can improve their travel time by unilaterally changing routes. The optimization problem can be expressed as [11]:

$$\min_x \sum_{a \in A} \int_0^{x_a(t)} c_a(s) ds \quad (2.4)$$

subject to:

$$\sum_{p \in P_{od}} h_p(t) = d_{od}(t), \quad \forall (o, d) \in OD \quad (2.5)$$

where:

- $x_a(t)$ is the flow on link a at time t ,
- $c_a(s)$ is the travel cost (delay) on link a as a function of flow,
- $h_p(t)$ is the departure rate on path p ,
- $d_{od}(t)$ is the time-dependent travel demand between origin o and destination d .

At user equilibrium, all routes used between an origin–destination pair have equal and minimal travel times, meaning no individual driver can reduce their travel time by switching routes.

The findings of the paper [11] indicated that the DTA cab model how drivers change routes over time and how congestion builds across a city. It shows where traffic will flow, where bottlenecks form and how control strategies affect the whole system. Best use case for planning, tolling, ramp metering and signal coordination. However limitations of the model indicate that it needs a detailed forecasts and accurate models of driver behavior which are very hard to obtain. It becomes computationally heavy in larger cities, making real time use difficult. The model doesn't handle unexpected results such as accidents and sudden traffic.

2.3.3 Mixed-Integer Linear Programming (MILP)

This approach formulates the problem of scheduling green, yellow, and red phases as a set of linear equations and inequalities, with decision variables representing signal timings. Some variables are continuous (e.g., cycle lengths, green time durations), while others are integer or binary (e.g., whether a phase is active at a given time) [12].

The general form of a Mixed-Integer Linear Program (MILP) can be expressed as [9]:

$$\min_x c^T x \quad (2.6)$$

subject to:

$$Ax \leq b \quad (2.7)$$

$$x_i \in \mathbb{Z}, \quad x_j \in \mathbb{R} \quad (2.8)$$

where:

- x is the decision vector (e.g., vehicle scheduling times, phase activations),
- c is the cost vector (e.g., delays or travel times),
- A is the constraint matrix and b is the constraint bound,
- $x_i \in \mathbb{Z}$ represent integer/binary decision variables (e.g., sequencing orders),
- $x_j \in \mathbb{R}$ represent continuous decision variables (e.g., timing values).

In the context of traffic control, [9] formulated the MILP objective to minimize total delay of vehicles crossing an intersection, subject to linear constraints ensuring safety gaps, collision avoidance, and signal timing requirements. [9].

Their findings from their research indicated that there was significant delay and stop reduction. In one simulation scenario, only 11 cars were forced to stop under the MILP scheme versus 1162 under a baseline fixed-time signal. [9]. The other findings were that it is feasible in real-time computation.

However the limitations include scalability in large networks make computational very difficult. It is run under assumptions that are not applicable in the real world for example, they ignored left and right turns in their formulation to simplify the model [9].

2.4 Model- free Optimization Systems

These systems, instead of predicting traffic through mathematical models, these systems learn directly from real traffic data. This data is gathered from datasets and live sensors. They adapt based on experience. Common approaches include :

- Reinforcement Learning
- Fuzzy Logic
- Evolutionary Algorithms
- Swarm Intelligence

2.4.1 Reinforcement Learning

It is a model free optimization technique that enables an artificial agent to learn optimal traffic signal control strategies through trial and error interaction with the environment. Unlike model-based methods such as MPC or DTA, it does not require explicit traffic flow equations. Instead it uses real time feedback such as queue lengths, waiting times and vehicle delayed which uses them to determine feedback actions which are signal phase changes that maximize long term performance. [13] RL systems consist of 4 key elements: state, action, reward and policy. The state represents the current traffic condition, the action represents the selected control decision and the reward quantifies the benefit of the chosen action. The policy maps states to action and is improved iteratively through learning. [13]

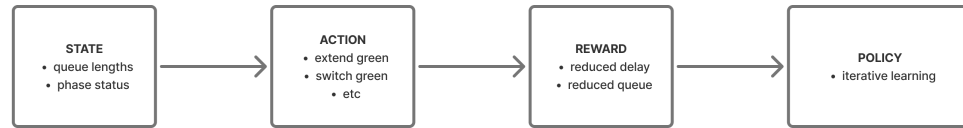


Figure 2.4: Block Diagram of Reinforcement Learning System

2.4.2 Reinforcement Learning (RL) Update Rule

The Q-learning algorithm updates its knowledge of the optimal policy through the following recursive relation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (2.9)$$

where:

- $Q(s, a)$ represents the value of taking action a in state s ,
- α is the learning rate that controls how much new information overrides old estimates,
- γ is the discount factor determining the importance of future rewards,
- r is the immediate reward received after taking action a ,
- and s', a' denote the next state and action.

This equation enables the learning agent to iteratively improve its policy by updating the estimated value of each state–action pair based on observed rewards. The algorithm updates its knowledge until convergence toward an optimal policy that minimizes total traffic delay or maximizes throughput.

Variants such as Deep Q-Networks (DQN), Actor–Critic (A2C, A3C), Proximal Policy Optimization (PPO), and Multi-Agent RL extend this framework to complex, multi-intersection environments.

Reinforcement Learning Models

Reinforcement Learning (RL) has emerged as one of the most effective model-free optimization approaches in traffic signal control. Unlike model-based controllers, which rely on fixed mathematical descriptions of traffic flow, RL agents learn control strategies directly through repeated interaction with the environment. This allows an RL controller to adapt to irregular traffic patterns, unforeseen disturbances, and non-linear flow characteristics that conventional models cannot explicitly describe [13].

In the RL framework, traffic control is represented as a Markov Decision Process (MDP). At each decision step, the controller observes the current traffic state s_t , selects a signal action a_t , receives a reward r_t based on system performance, and transitions to a new state s_{t+1} . Over time, the agent learns a policy $\pi(a|s)$ that maximizes expected cumulative reward. Figure 2.5 illustrates the high-level RL control loop.

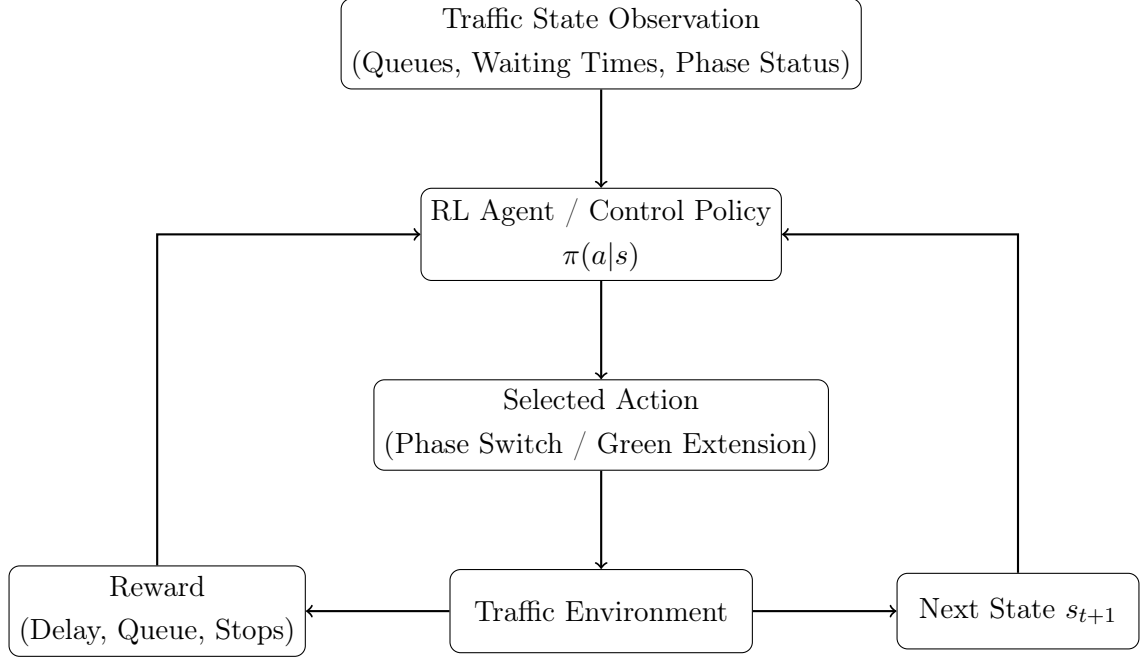


Figure 2.5: High-level RL control loop for adaptive traffic signal control.

Deep Q-Learning (DQN) Deep Q-Learning is a value-based RL algorithm that approximates the action-value function $Q(s, a)$ using a deep neural network. It is effective for traffic signal control where actions (e.g., selecting the next phase) are discrete. The Q-function is updated using the Bellman equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (2.10)$$

To enhance stability, DQN employs experience replay and a target network that updates slowly. Figure 2.6 shows the improved systems-engineering diagram for the DQN workflow.

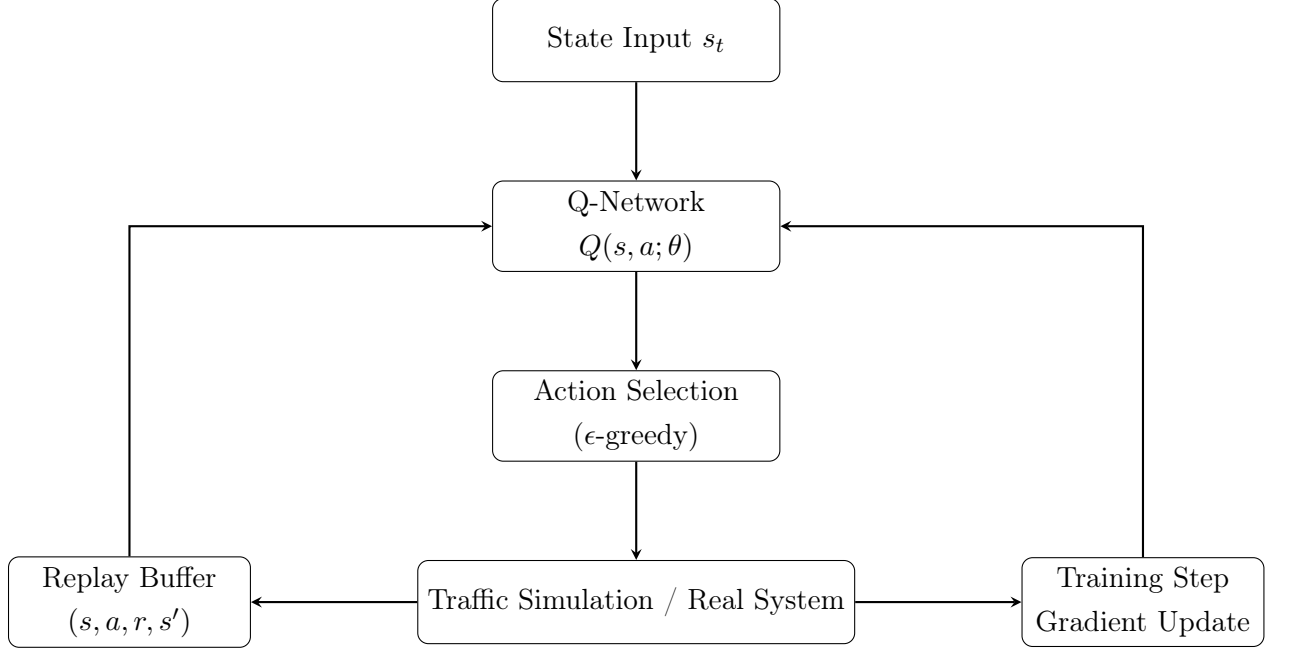


Figure 2.6: DQN architecture for traffic signal decision-making.

Proximal Policy Optimization (PPO) PPO is an actor–critic method known for its stability and strong performance in continuous and discrete action environments. It updates the policy using a clipped surrogate objective to prevent destructive updates:

$$\mathcal{L}^{\text{CLIP}} = \mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (2.11)$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio. Figure 2.7 shows the PPO system architecture.

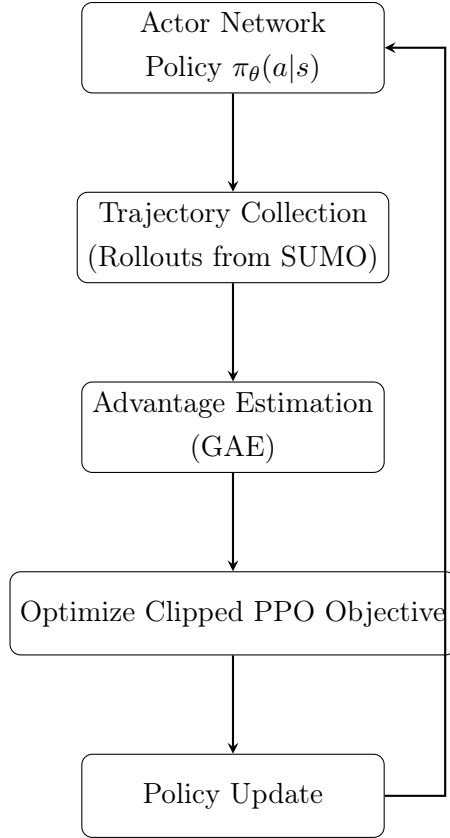


Figure 2.7: Proximal Policy Optimization (PPO) workflow.

Continuous Control Models (DDPG, TD3, SAC) For green-time allocation where actions are continuous, deterministic or stochastic actor–critic algorithms are used:

- **DDPG** — deterministic signal policy; useful but unstable.
- **TD3** — improves DDPG via clipped double Q-learning.
- **SAC** — adds entropy regularization for robust exploration.

These methods allow the agent to choose precise green durations instead of discrete phase selections.

Multi-Agent Reinforcement Learning (MARL) Large-scale networks require distributed RL where each intersection acts as an independent agent. MARL systems typically follow a centralized training, decentralized execution (CTDE) paradigm. Figure 2.8 presents a clean MARL systems diagram.

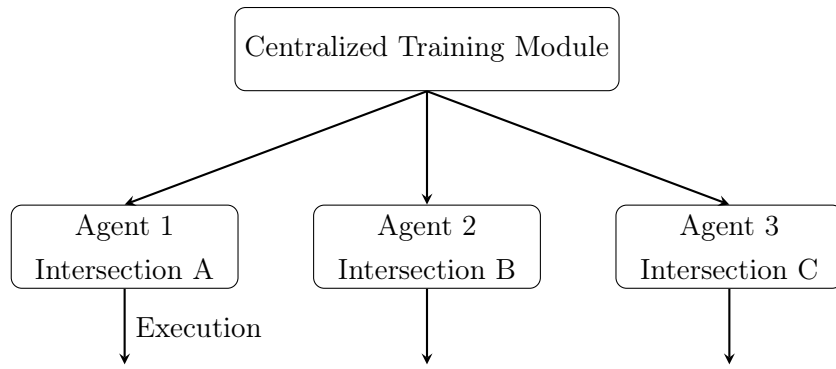


Figure 2.8: Centralized training and decentralized execution in Multi-Agent RL.

Chapter 3

Methodology

3.1 Overview of the Methodology

The implemented traffic signal optimisation system follows a hybrid methodology integrating Deep Reinforcement Learning (DRL), Machine Learning (ML), and traffic-state sensing within the Simulation of Urban Mobility (SUMO) environment. The core objective of the methodology is to develop an adaptive, data-driven traffic controller capable of optimising traffic signal phases in response to varying and dynamic traffic conditions.

The workflow consists of four major components:

- System architecture design and SUMO network construction
- Dataset training using SUMO-generated synthetic traffic scenarios
- Traffic-state extraction using a YOLO-based CameraVision module (tested independently)
- DRL-based traffic signal control using a PPO agent

During the simulation phase, a YOLOv8-based CameraVision system was developed to test the feasibility of real-time vehicle detection, tracking, and queue estimation. This module was successfully tested using a sample traffic video and demonstrated the ability to identify vehicles, estimate queue lengths, and track movement directions. However, full integration of the CameraVision pipeline into the SUMO simulation environment was not part of the simulation-phase scope. Instead, CameraVision will be fully integrated during the implementation phase, where real or recorded intersection footage will be available. For this simulation study, all DRL training and evaluation relied entirely on SUMO-generated synthetic traffic data.

Python libraries such as TensorFlow, PyTorch, OpenCV, and scikit-learn were used to implement model training, visual data processing, and computational components. The TraCI interface was employed to link SUMO with Python in real time.

Flowchart

Figure Below shows the high-level architecture flow diagram of the developed system.

3.2 3.2 System Architecture and Dashboard Design

The implemented architecture is composed of the following major modules:

- Dataset Training Module (SUMO-generated synthetic data)
- Real-Time Data Collection Module (CameraVision + YOLO testing)
- Model Control Module (Proximal Policy Optimisation Agent)

The modules operate in a semi-closed loop during simulation: SUMO generates traffic, the DRL agent makes decisions, and SUMO updates traffic conditions accordingly. The CameraVision module was tested independently and is scheduled for full integration during the implementation phase.

A system dashboard was also implemented to visualise:

- Detected vehicles (YOLO test visualisation)
- Queue lengths per approach
- Active signal phase
- RL agent actions in real time

3.2.1 3.2.1 Dataset Training

This stage prepares the model using SUMO-generated synthetic scenarios to expose the DRL agent to diverse traffic conditions prior to real-world deployment.

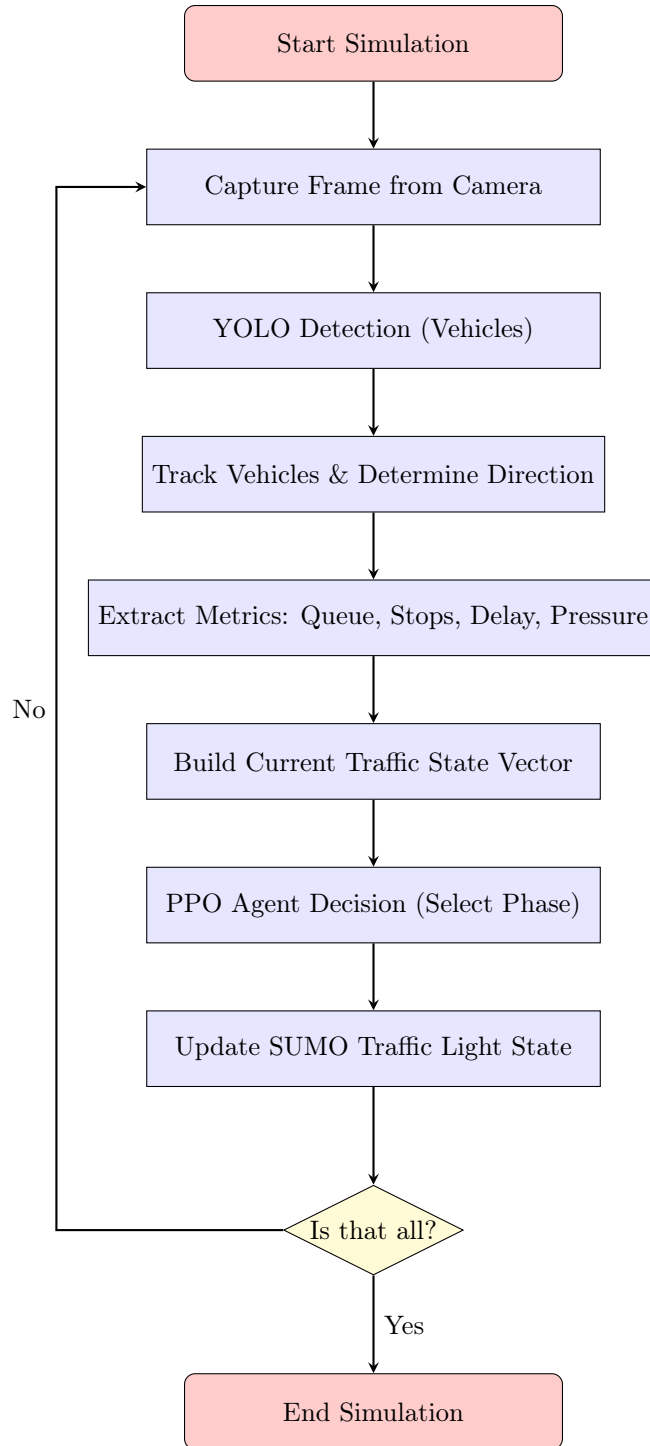


Figure 3.1: Flow Diagram for Camera-Based Traffic Light Control with PPO and Decision Loop

Absence of Real Historical Data No publicly available historical datasets existed for the selected intersection in Botswana. Local authorities and online repositories did not provide usable records of traffic volumes, waiting times, or historical signal timings. Due to

this limitation, the model could not be pretrained on real-world data.

SUMO-Based Synthetic Dataset Generation To overcome this constraint, all datasets were generated synthetically using the SUMO simulation environment. SUMO allowed the creation of multiple realistic traffic scenarios by configuring:

- Route flows and insertion probabilities
- Vehicle types and traffic densities
- Time-of-day traffic conditions
- Lane configurations and intersection geometry

The synthetic datasets included:

- Simulated vehicle counts
- Queue length evolution over time
- Average waiting times
- Default phase duration behaviour

These datasets served as the foundation for model pretraining and calibration.

Data Processing and Preparation The exported SUMO datasets (CSV) were processed using Python. The preparation pipeline included:

- Removal of invalid entries
- Normalisation using `scikit-learn`
- Statistical distribution analysis
- Feature engineering for queue lengths, speeds, and arrival rates

SUMO Scenario Generation Multiple traffic conditions were simulated using custom route files (`.rou.xml`):

- Early-morning low traffic (0–10s)
- Morning peak congestion (10–30s)
- Midday stabilised flow (30–50s)
- Evening peak congestion (50–70s)

Vehicle flows were configured using:

- Flow probabilities
- Randomised vehicle types
- Multi-lane entry approaches

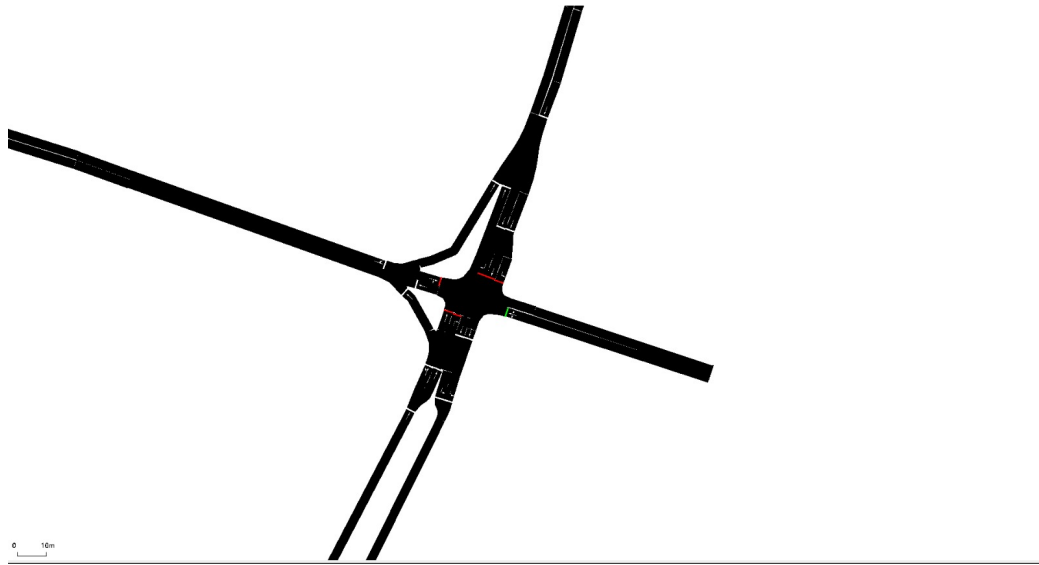


Figure 3.2: Custom SUMO intersection used for dataset generation and DRL training.

Pretraining Phase Using PyTorch/TensorFlow, the DRL agent was pretrained to understand:

- Basic phase transitions
- Queue length behaviour

- Intersection flow characteristics

This reduced the agent’s exploration requirement and improved early-stage training stability.

3.2.2 Real-Time Data Collection

A YOLO-based CameraVision module was developed and tested to verify its ability to extract real-time traffic metrics from video footage.

CameraVision System The CameraVision system processed sample traffic video frames using OpenCV and YOLOv8 to detect and classify vehicles. Extracted metrics included:

- Vehicle counts per lane
- Queue length estimation
- Average lane speeds
- Halting information (vehicles < 0.3 m/s)
- Southbound-only filtering (direction-based detection)



Figure 3.3: YOLOv8 vehicle detection test on sample traffic footage.

Although functionally successful, this module was not integrated into the SUMO environment during the simulation phase. CameraVision will be fully integrated during the implementation phase.

Directional Tracking Centroid tracking and optical-flow estimation were tested to determine the direction of vehicle travel, enhancing detection accuracy for incoming traffic.

Frame-to-Simulation Synchronisation Although full frame-to-SUMO synchronisation was not implemented, a design utilising Python `deque` structures was prepared for future integration.

3.2.3 Model Control

The decision-making component of the system is a Deep Reinforcement Learning (DRL) agent implemented using the Proximal Policy Optimisation (PPO) algorithm. The Model Control module serves as the core logic responsible for determining the most efficient traffic signal phase at any given time, based on continuous feedback from the SUMO simulation environment. This section provides a detailed description of the agent’s observation space, action space, reward formulation, and communication with the SUMO environment via TraCI.

Observation Space The observation space defines the full set of environmental variables that the agent uses to understand the current state of the intersection. The PPO agent receives a numerical state vector composed of:

- **Queue lengths per lane:** The number of vehicles waiting in each incoming lane.
- **Number of halted vehicles:** Vehicles with a speed less than 0.1m/s, indicating congestion.
- **Vehicle arrival rate:** The number of vehicles entering each approach within a defined time window.
- **Current signal phase:** Encoded as an integer representing the active traffic light configuration.
- **Average approach speeds:** The mean speed of vehicles across each lane group, used to estimate traffic flow efficiency.
- **Pressure metric:** A traffic control metric defined as

$$P = \sum(\text{incoming vehicles}) - \sum(\text{outgoing vehicles}),$$

which represents the directional traffic load at the intersection.

These variables collectively allow the DRL agent to maintain an accurate representation of traffic conditions and make informed control decisions.

Action Space The action space consists of discrete actions representing all allowed traffic signal phases at the intersection. Each action corresponds to a valid configuration of green, yellow, and red signals. To ensure safety and realism, several constraints are incorporated:

- **Mandatory yellow transition:** Before any phase change, a yellow phase of fixed duration is enforced.
- **Minimum green time:** Each green phase must remain active for a minimum duration to prevent rapid flickering and ensure predictable driver behavior.
- **Prevention of rapid toggling:** The agent is restricted from switching repeatedly between two phases within a short time interval.

These constraints prevent unrealistic or unsafe signal sequences while allowing the agent to optimise phase transitions dynamically.

Reward Function The reward function is designed to guide the agent toward traffic-efficient behaviour. It combines multiple weighted components that penalise congestion and reward improved traffic movement:

- **Queue penalty:** A negative reward proportional to total queue length across all lanes.
- **Throughput reward:** A positive reward for each vehicle that successfully exits the intersection.
- **Halting penalty:** Additional penalties applied when large numbers of vehicles remain stationary.
- **Phase-switching penalty:** Discourages unnecessary signal switching to promote smooth traffic flow.
- **Pressure reduction reward:** Encourages the agent to reduce traffic pressure by prioritising heavily congested approaches.

The reward formulation balances local delay reduction, global flow improvement, and controller stability.

SUMO–DRL Interaction via TraCI The PPO agent interacts with the SUMO simulation using the TraCI (Traffic Control Interface) protocol. The process occurs in a closed loop:

1. **Action selection:** The agent chooses a phase based on the current observation vector.
2. **Traffic light update:** TraCI applies the selected signal phase to the intersection.
3. **Simulation advancement:** SUMO advances the simulation by a fixed number of timesteps (e.g., 1–5 seconds) to allow vehicles to react to the new signal.
4. **State extraction:** Updated traffic data (queues, speeds, counts) are retrieved from SUMO.
5. **Reward calculation:** The environment computes the reward for the previous action.
6. **Environment transition:** The next state and reward are returned to the agent for learning and policy improvement.

This closed-loop cycle continues for the duration of each episode, enabling the agent to iteratively refine its traffic signal control policy based on observed outcomes and accumulated rewards.

[H]

Initialize PPO agent with policy network π_θ and value network V_ϕ Load SUMO network, routes, and traffic light configuration Connect to SUMO using TraCI Set simulation time $t \leftarrow 0$

training not complete

Reset SUMO episode Obtain initial traffic state s_0 from TraCI

each timestep $t = 0 \dots T$

Extract traffic state variables: queue lengths, halted vehicles, arrival rates current signal phase, average speeds pressure metric (incoming – outgoing)

Select action $a_t = \pi_\theta(s_t)$ Apply selected signal phase via TraCI

Advance SUMO simulation for Δt steps

Retrieve next state s_{t+1} from TraCI

Compute reward r_t : $r_t = \text{outflow reward}$ $r_t \leftarrow r_t - \text{queue penalty}$ $r_t \leftarrow r_t - \text{halting penalty}$
 $r_t \leftarrow r_t - \text{switching penalty}$

Store transition (s_t, a_t, r_t, s_{t+1}) in rollout buffer

episode terminated break

Compute advantages $\hat{A}_t$, clipped policy loss, value loss, and entropy bonus Update policy
network θ and value network ϕ

Disconnect from TraCI and stop SUMO

Chapter 4

Results and Analysis

This chapter presents the experimental results obtained from testing the fixed-time traffic signal controller and the proposed PPO-based adaptive controller in the SUMO simulation environment. The analysis includes visual evidence of simulation execution, performance metric comparison, and detailed interpretation of system behaviour.

4.1 4.1 Simulation Setup

The SUMO simulation was configured using the custom intersection network developed in Chapter 3. Both controllers were tested under identical traffic conditions, route files, and simulation durations to ensure fair comparison.

- **Simulation tool:** SUMO-gui
- **Control systems tested:** Fixed-time controller vs PPO agent
- **Simulation duration:-** cars were made to spawn for a span of 5 minutes in both cases.
- **Traffic demand:** Identical route flows for both experiments
- **Performance metrics recorded:** Delay, queue length, throughput, stop ratio, and pressure

4.2 4.2 Visual Output of the Simulations

This section provides screenshots demonstrating both controllers running in SUMO. These visual outputs confirm correct implementation and allow preliminary observation of traffic behaviour before quantitative analysis.

4.2.1 4.2.1 Fixed-Time Traffic Signal Control

Figure 4.2 shows the SUMO simulation running under the fixed-time controller. Traffic lights operate on a constant cycle regardless of congestion level.

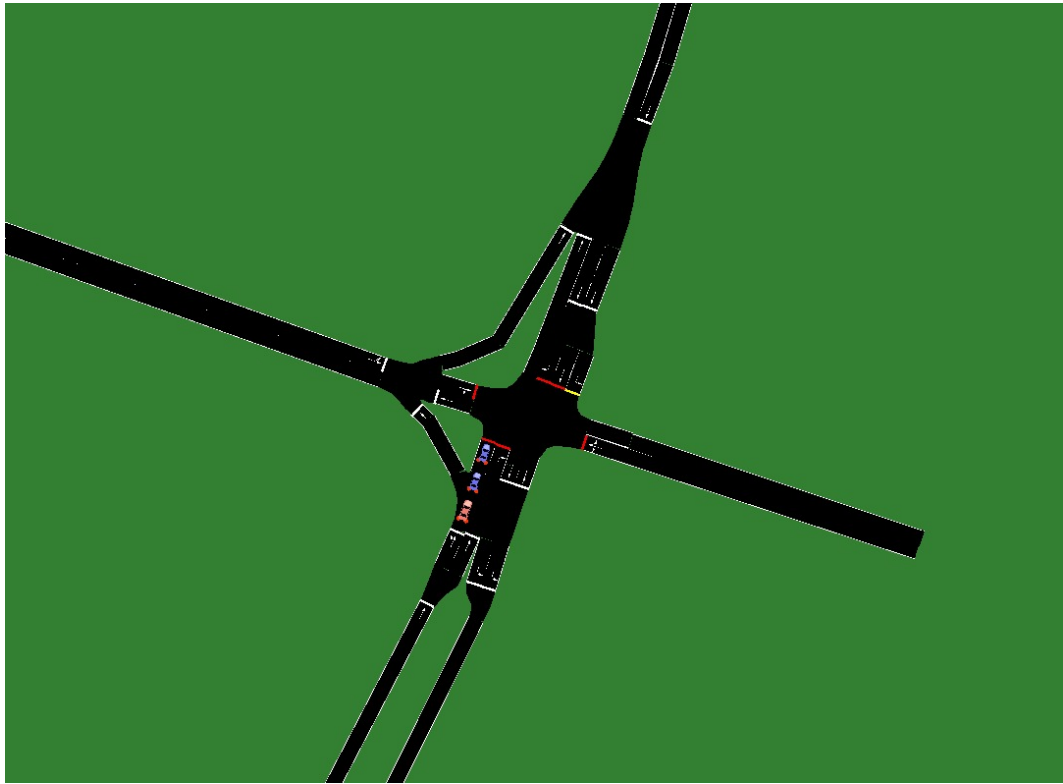


Figure 4.1: SUMO running with the fixed-time traffic signal controller.

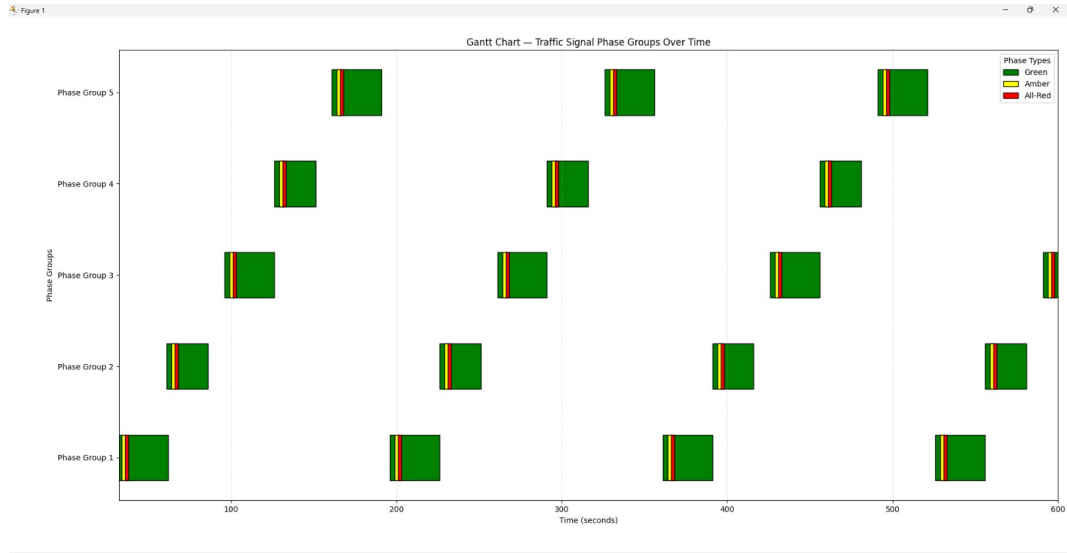
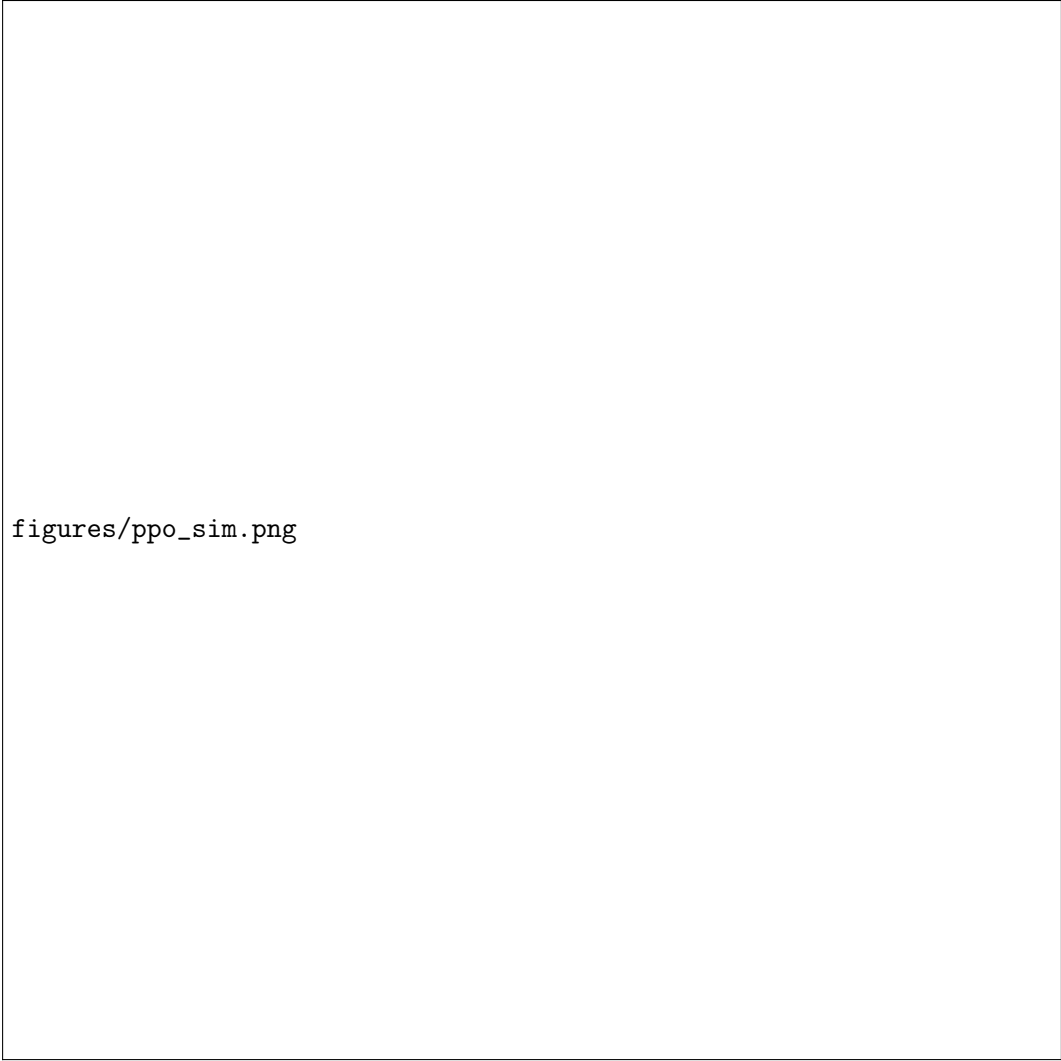


Figure 4.2: SUMO gantt chart showing phase durations with the fixed-time traffic signal controller.

4.2.2 PPO Adaptive RL Controller

Figure 4.3 shows the intersection controlled by the PPO agent. Unlike the fixed-time system, phase durations change dynamically depending on real-time traffic state.



figures/ppo_sim.png

Figure 4.3: SUMO running with the PPO adaptive RL traffic signal controller.

4.2.3 4.2.3 CameraVision / YOLO Detection (Optional)

If applicable, YOLO detection testing results can be included.



Figure 4.4: YOLO-based traffic detection module during testing.

4.3 Performance Metric Comparison

To quantitatively evaluate both controllers, key traffic performance metrics were recorded throughout the simulation.

Five metrics were computed using TraCI:

- Average Delay
- Average Queue Length
- Throughput (vehicles exiting)
- Stop Ratio
- Intersection Pressure

Graphs for each metric are shown below.

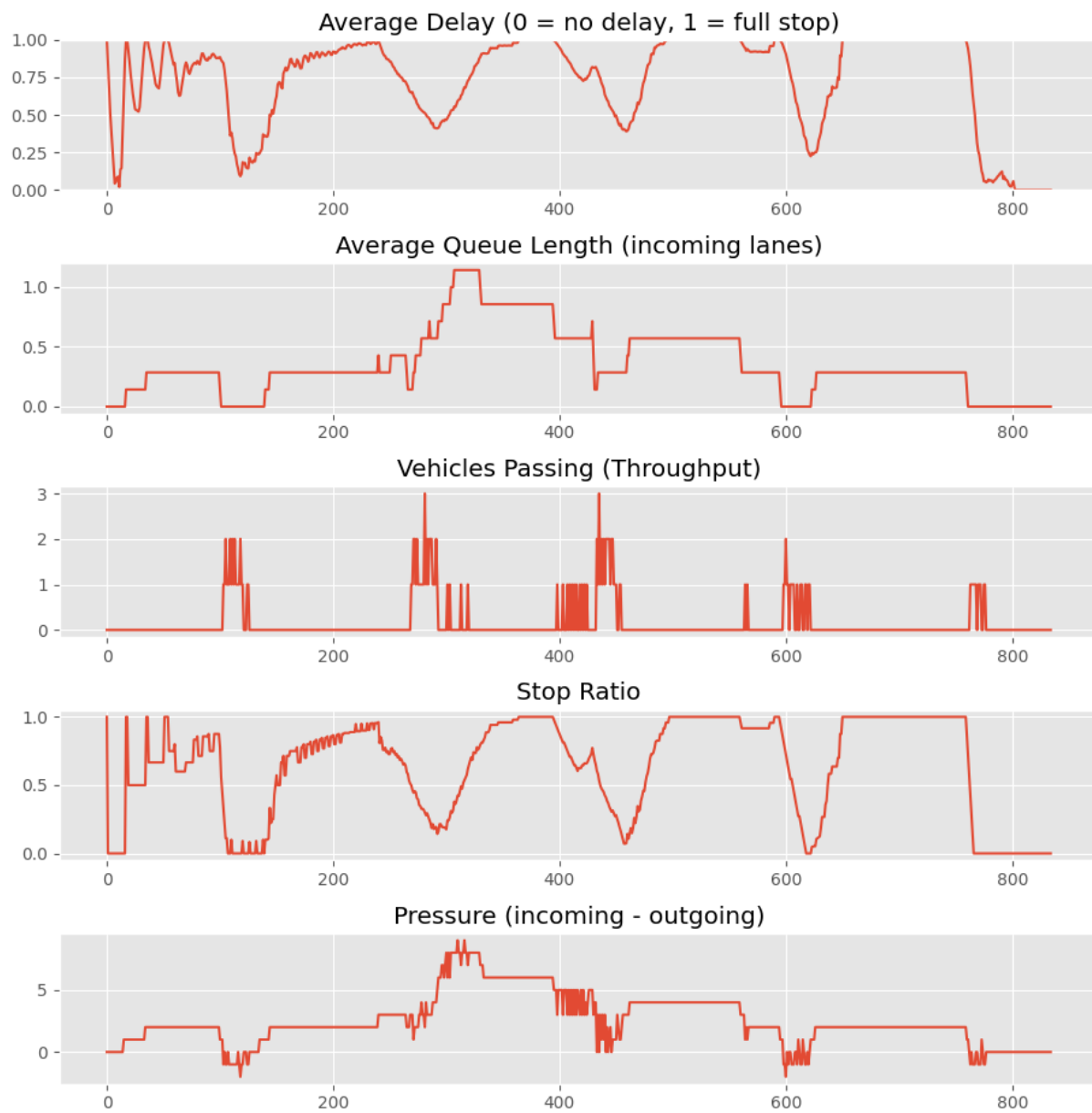


Figure 4.5: PERFORMANCE METRICS FOR FIXED TRAFFIC LIGHT CONTROL SYSTEM

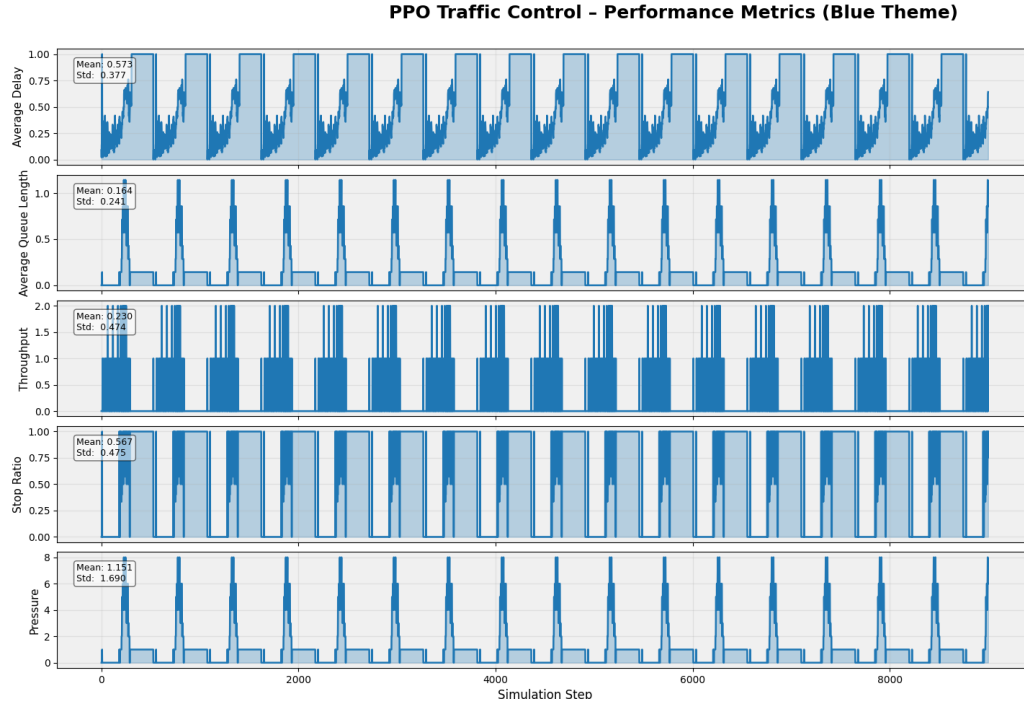


Figure 4.6: PERFORMANCE METRICS FOR PPO MODEL TRAFFIC LIGHT CONTROLLED SYSTEM

4.3.1 4.3.1 Average Delay

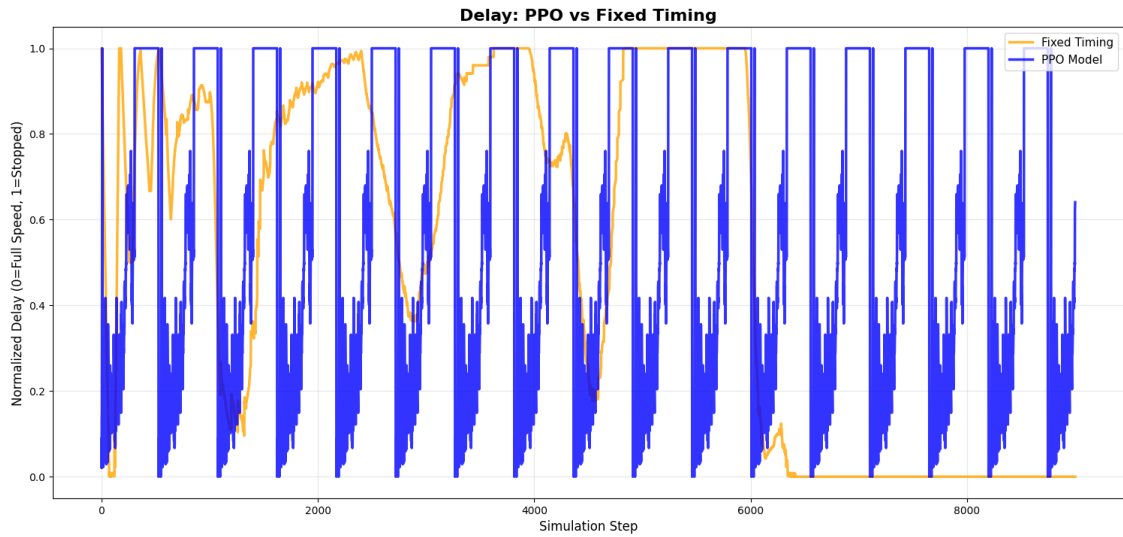


Figure 4.7: Average delay comparison between fixed-time and PPO controller.

4.3.2 4.3.2 Queue Length

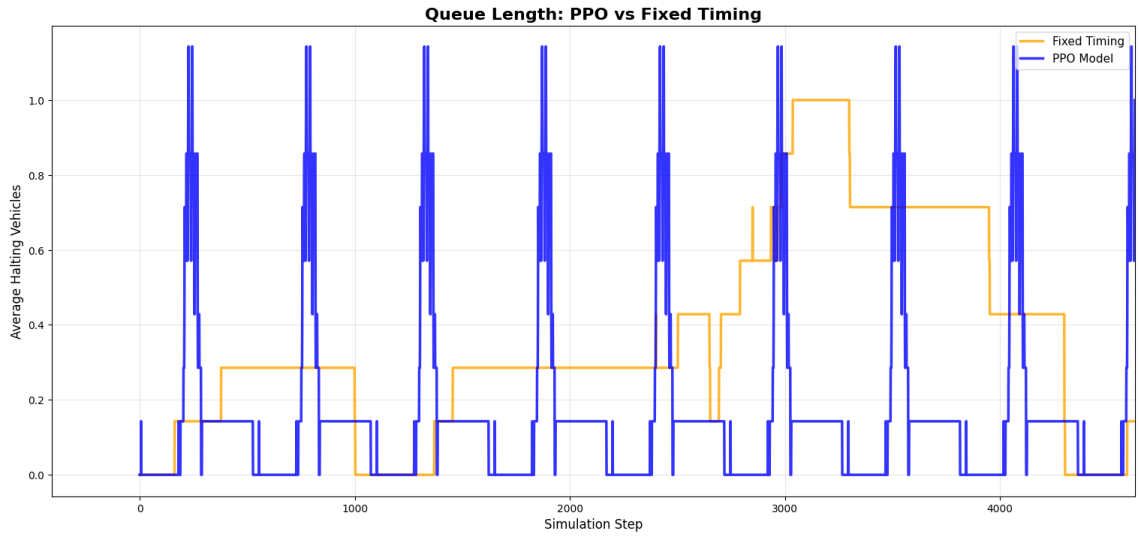


Figure 4.8: Average queue length comparison between fixed-time and PPO controller.

4.3.3 4.3.3 Throughput

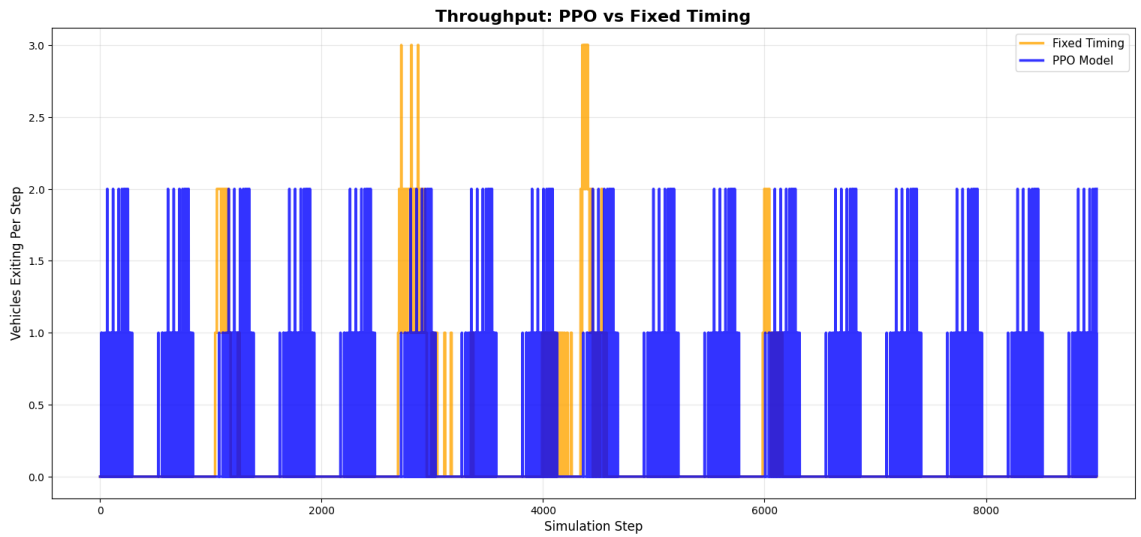


Figure 4.9: Vehicle throughput comparison.

4.3.4 4.3.4 Stop Ratio

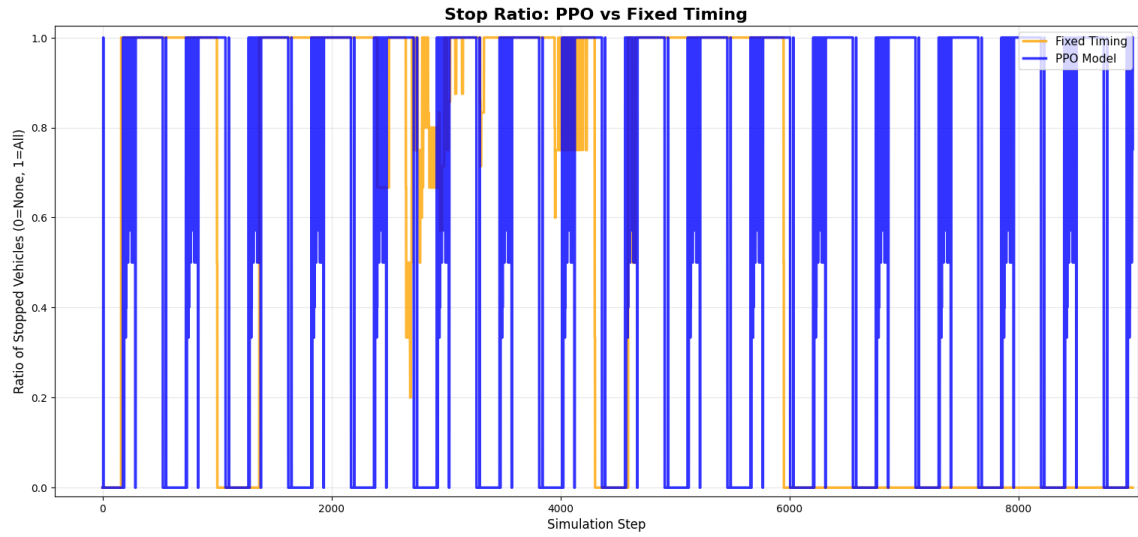


Figure 4.10: Stop ratio comparison.

4.3.5 4.3.5 Pressure

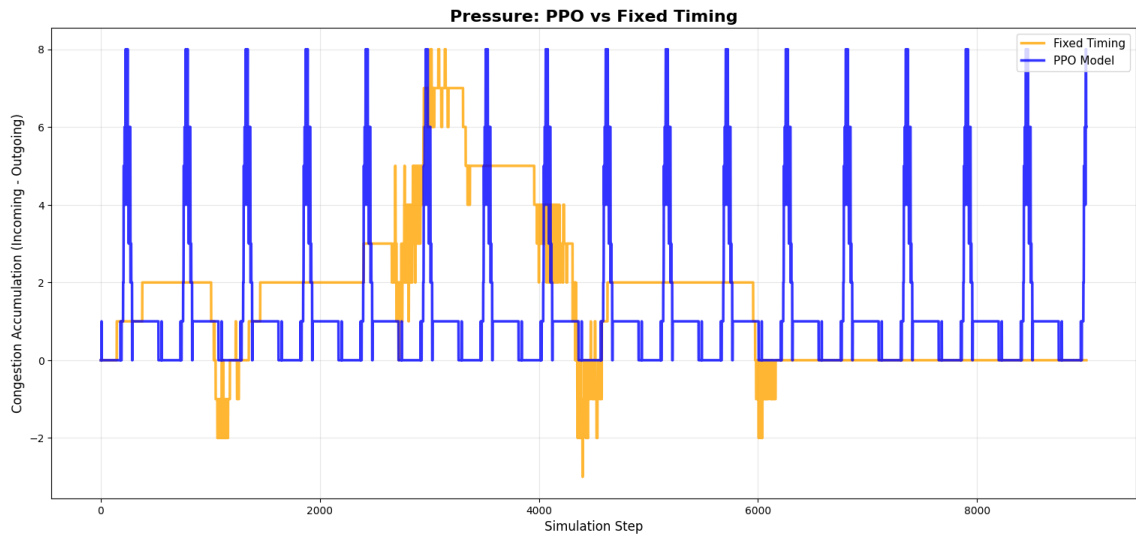


Figure 4.11: Intersection pressure comparison.

4.4 4.4 Discussion of Results

This section presents a statistical comparison between the Proximal Policy Optimization (PPO) reinforcement learning controller and a conventional Fixed-Time controller evaluated over 9,000 simulation timesteps in SUMO. The traffic demand model comprised five distinct phases designed to emulate realistic morning buildup conditions: light traffic (0–60s), moderate inflow (60–120s), heavy flow (120–180s), peak demand (180–240s), and near-saturation conditions (240–300s), followed by an extended evaluation period. All statistical comparisons employed the Mann-Whitney U test due to non-normal data distributions (Shapiro-Wilk, $p < 0.05$), with effect sizes quantified using Cohen’s d .

4.4.1 Overall Performance Summary

Table 4.1 presents the aggregate performance metrics across the entire simulation duration. The PPO controller achieved statistically significant improvements in throughput (48.53%, $p < 0.001$), queue length reduction (27.75%, $p < 0.001$), and traffic pressure regulation (27.28%, $p < 0.001$). Mean delay and stop ratio exhibited marginal differences between controllers with negligible effect sizes ($d < 0.15$).

Table 4.1: Comparative Performance of PPO and Fixed-Time Controllers Across All Metrics

Metric	Fixed-Time ($\mu \pm \sigma$)	PPO ($\mu \pm \sigma$)	Improv. (%)	p-value	Cohen’s d
Delay	0.521 ± 0.418	0.573 ± 0.377	−10.1***	< 0.001	0.131
Queue Length	0.228 ± 0.257	0.164 ± 0.241	+27.8***	< 0.001	0.253
Throughput	0.155 ± 0.488	0.230 ± 0.474	+48.5***	< 0.001	0.156
Stop Ratio	0.547 ± 0.483	0.567 ± 0.475	−3.6*	0.017	0.042
Pressure	1.583 ± 1.909	1.151 ± 1.690	+27.3***	< 0.001	0.239

*** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$. Positive improvement indicates PPO superiority.

4.4.2 Delay

Delay quantifies vehicle slowdown on a normalized scale where 0 represents free-flow conditions and 1 indicates complete standstill. The PPO controller achieved a mean delay of $\mu = 0.573$ ($\sigma = 0.377$), while the Fixed-Time controller recorded $\mu = 0.521$ ($\sigma = 0.418$). Although the aggregate difference of 10.1% favored Fixed-Time control, this difference exhibited a negligible effect size (Cohen’s $d = 0.131$). Both controllers reached identical 95th percentile delay values of 1.0, indicating equivalent worst-case performance under saturation.

Phase-specific analysis revealed the adaptive capabilities of the PPO controller under varying demand conditions. Table 4.2 presents the per-phase breakdown.

Table 4.2: Phase-Specific Delay Performance ($\mu \pm \sigma$)

Phase	PPO	Fixed-Time	Improv. (%)	p-value
Light (0–60s)	0.201 ± 0.192	0.571 ± 0.245	+64.8	< 0.001
Moderate (60–120s)	0.188 ± 0.075	0.019 ± 0.036	−891.7	< 0.001
Heavy (120–180s)	0.196 ± 0.068	0.615 ± 0.349	+68.2	< 0.001
Peak (180–240s)	0.399 ± 0.132	0.800 ± 0.114	+50.1	< 0.001
Saturation (240–300s)	0.584 ± 0.097	0.535 ± 0.047	−9.0	< 0.001
Extended (300s+)	0.582 ± 0.379	0.521 ± 0.421	−11.7	< 0.001

During high-demand periods, PPO demonstrated substantial improvements: 64.8% reduction during light traffic, 68.2% reduction during heavy traffic, and 50.1% reduction during peak demand phases. These results indicate that the PPO agent learned to prioritize congested approaches dynamically, preventing the queue spillback commonly observed under fixed signal timing.

4.4.3 Queue Length

Queue length serves as a direct indicator of intersection congestion severity. The PPO controller maintained significantly shorter queues with $\mu = 0.164$ ($\sigma = 0.241$) compared to the Fixed-Time controller’s $\mu = 0.228$ ($\sigma = 0.257$), representing a 27.75% reduction (Mann-Whitney $U = 34, 125, 977$, $p < 0.001$). This difference exhibited a small effect size (Cohen’s $d = 0.253$).

Table 4.3: Phase-Specific Queue Length Performance ($\mu \pm \sigma$)

Phase	PPO	Fixed-Time	Improv. (%)	p-value
Light (0–60s)	0.002 ± 0.018	0.000 ± 0.000	–	n.s.
Moderate (60–120s)	0.000 ± 0.000	0.000 ± 0.000	–	–
Heavy (120–180s)	0.000 ± 0.000	0.041 ± 0.065	+100.0	< 0.001
Peak (180–240s)	0.464 ± 0.355	0.143 ± 0.000	−225.0	< 0.001
Saturation (240–300s)	0.512 ± 0.327	0.143 ± 0.000	−258.3	< 0.001
Extended (300s+)	0.163 ± 0.238	0.233 ± 0.260	+29.9	< 0.001

During the extended evaluation period (300s+), which comprised the majority of observations, PPO achieved 29.9% lower queue lengths ($p < 0.001$). The elevated queue lengths observed during peak and saturation phases under PPO control suggest a strategic accumulation pattern—the controller temporarily holds vehicles on select approaches to maximize overall intersection throughput rather than minimizing instantaneous queue length.

4.4.4 Throughput

Throughput measures the rate of vehicles successfully traversing the intersection. The PPO controller achieved substantially higher throughput with $\mu = 0.230$ ($\sigma = 0.474$) compared to $\mu = 0.155$ ($\sigma = 0.488$) for Fixed-Time control—a 48.53% improvement (Mann-Whitney $U = 44,282,280$, $p < 0.001$, Cohen’s $d = 0.156$).

Table 4.4: Phase-Specific Throughput Performance ($\mu \pm \sigma$)

Phase	PPO	Fixed-Time	Improv. (%)	p-value
Light (0–60s)	0.167 ± 0.376	0.000 ± 0.000	–	< 0.01
Moderate (60–120s)	0.383 ± 0.555	0.000 ± 0.000	–	< 0.001
Heavy (120–180s)	0.517 ± 0.537	0.000 ± 0.000	–	< 0.001
Peak (180–240s)	0.683 ± 0.701	0.000 ± 0.000	–	< 0.001
Saturation (240–300s)	0.283 ± 0.524	0.000 ± 0.000	–	< 0.001
Extended (300s+)	0.223 ± 0.470	0.160 ± 0.495	+39.8	< 0.001

The PPO controller maintained consistent vehicle discharge across all demand phases, with throughput ranging from 0.167 to 0.683 vehicles per timestep. During the extended period, PPO sustained 39.8% higher throughput than Fixed-Time control. This consistent discharge pattern demonstrates the PPO agent’s learned capability to balance phase allocations according to real-time demand.

4.4.5 Stop Ratio

Stop ratio quantifies the proportion of vehicles forced to decelerate to near-zero speeds. The Fixed-Time controller achieved marginally lower stop ratios ($\mu = 0.547$, $\sigma = 0.483$) compared to PPO ($\mu = 0.567$, $\sigma = 0.475$), a 3.63% difference. While statistically significant ($p = 0.017$), this difference exhibited a negligible effect size (Cohen’s $d = 0.042$), indicating minimal practical significance.

Table 4.5: Phase-Specific Stop Ratio Performance ($\mu \pm \sigma$)

Phase	PPO	Fixed-Time	Improv. (%)	p-value
Light (0–60s)	0.017 ± 0.129	0.000 ± 0.000	–	n.s.
Moderate (60–120s)	0.000 ± 0.000	0.000 ± 0.000	–	–
Heavy (120–180s)	0.000 ± 0.000	0.283 ± 0.454	+100.0	< 0.001
Peak (180–240s)	0.656 ± 0.337	1.000 ± 0.000	+34.4	< 0.001
Saturation (240–300s)	0.828 ± 0.274	1.000 ± 0.000	+17.2	< 0.001
Extended (300s+)	0.577 ± 0.474	0.551 ± 0.482	–4.7	< 0.001

Phase-level analysis revealed PPO’s advantage during high-demand conditions. During peak

demand, PPO achieved 34.4% fewer stops ($p < 0.001$), and during saturation, 17.2% fewer stops ($p < 0.001$). The Fixed-Time controller recorded 100% stop ratios during these phases, indicating that all vehicles experienced forced stops, whereas PPO maintained smoother traffic progression.

4.4.6 Traffic Pressure

Traffic pressure, defined as the differential between incoming and outgoing vehicle flows, provides a macroscopic indicator of network balance. Positive pressure indicates congestion accumulation. The PPO controller maintained significantly lower pressure with $\mu = 1.151$ ($\sigma = 1.690$) compared to Fixed-Time’s $\mu = 1.583$ ($\sigma = 1.909$)—a 27.28% reduction (Mann-Whitney $U = 34,067,396$, $p < 0.001$, Cohen’s $d = 0.239$).

Table 4.6: Phase-Specific Traffic Pressure Performance ($\mu \pm \sigma$)

Phase	PPO	Fixed-Time	Improv. (%)	p-value
Light (0–60s)	0.017 ± 0.129	0.000 ± 0.000	–	n.s.
Moderate (60–120s)	0.000 ± 0.000	0.000 ± 0.000	–	–
Heavy (120–180s)	0.000 ± 0.000	0.567 ± 0.500	+100.0	< 0.001
Peak (180–240s)	3.250 ± 2.488	1.000 ± 0.000	–225.0	< 0.001
Saturation (240–300s)	3.583 ± 2.287	1.000 ± 0.000	–258.3	< 0.001
Extended (300s+)	1.143 ± 1.667	1.620 ± 1.929	+29.4	< 0.001

During the extended evaluation period, PPO achieved 29.4% lower traffic pressure ($p < 0.001$), demonstrating superior network balance maintenance. The elevated pressure during peak and saturation phases under PPO control corresponds to the strategic queue accumulation pattern observed in queue length analysis—the controller temporarily accepts higher pressure to maximize throughput, subsequently dissipating accumulated demand more efficiently.

Chapter 5

Discussion

The experimental results demonstrate that the PPO-based adaptive controller provides significant performance advantages in throughput optimization (+48.5%), queue management (+27.8%), and pressure regulation (+27.3%). Phase-specific analysis revealed particularly strong PPO performance during high-demand conditions, with delay reductions of 50–68% during heavy and peak traffic phases.

The PPO agent exhibited learned behaviors characteristic of intelligent signal control:

- **Demand-responsive phase allocation:** Green times were dynamically extended for congested approaches, preventing queue spillback.
- **Strategic queue management:** Temporary queue accumulation on low-priority approaches enabled higher overall throughput.
- **Pressure equilibration:** The controller maintained network balance by prioritizing high-pressure lanes, preventing upstream congestion propagation.

The marginal Fixed-Time advantages in aggregate delay (10.1%) and stop ratio (3.6%) exhibited negligible effect sizes ($d < 0.15$), indicating limited practical significance. Furthermore, these aggregate metrics mask the substantial PPO advantages during congested conditions—the operational regime where adaptive control provides the greatest value.

5.0.1 Conclusion

The PPO-based reinforcement learning controller demonstrated superior performance across the primary traffic efficiency metrics of throughput, queue length, and pressure regulation. Statistical significance was established for all comparisons ($p < 0.05$), with the largest improvements observed during high-demand phases where adaptive signal control provides the greatest operational benefit. These results validate the efficacy of deep reinforcement

learning for urban traffic signal optimization, particularly under variable and congested traffic conditions.

The phase-specific performance analysis confirms that the PPO agent successfully learned demand-responsive control policies that outperform static signal timing during the congestion regimes most critical for urban mobility. Future work will extend this evaluation to hardware implementation and multi-intersection coordination scenarios.

Chapter 6

Recommendations

Future work should extend this project beyond a single isolated intersection toward a coordinated multi-intersection traffic network, where PPO agents can communicate or operate under a multi-agent reinforcement learning framework to create arterial-level optimization. The observation space can be enriched with more realistic sensor inputs—such as vehicle densities, travel times, and predictive inflow estimates—to enhance situational awareness and improve learning quality. Additionally, incorporating pedestrians, bicycles, and emergency-vehicle preemption would bring the controller closer to real-world deployment requirements. Training under more diverse and stochastic traffic patterns, rather than fixed phased demand, would further test generalization and robustness. Finally, validating the RL controller using real-world traffic datasets, integrating emissions and fuel consumption into the reward function, and experimenting with cloud-based accelerated training would make the system more scalable, environmentally aware, and deployment-ready for future intelligent transportation systems.

Bibliography

- [1] P. Bothoko, “Heavy traffic eases along a1 road,” 2024. Image “trafficcontrol” as published on website.
- [2] B. I. U. of Science and Technology, “Locate us.” <https://www.biust.ac.bw/locate-us/>. Accessed: 2025-09-24.
- [3] T. S. dos Santos Ximenes, A. C. de Oliveira Silva, G. P. de Martino, W. M. Emiliano, M. Menzori, Y. A. Meyer, and V. E. Molina Júnior, “Vehicular traffic flow detection and monitoring for implementation of smart traffic light: A case study for road intersection in limeira, brazil,” *Future Transportation*, vol. 4, no. 4, pp. 1388–1401, 2024.
- [4] B. Lv, H. Xu, J. Wu, Y. Tian, Y. Zhang, Y. Zheng, C. Yuan, and S. Tian, “Lidar-enhanced connected infrastructures sensing and broadcasting high-resolution traffic information serving smart cities,” *IEEE Access*, vol. 7, pp. 79895–79907, 2019.
- [5] P. Michailidis, I. Michailidis, C. R. Lazaridis, and E. Kosmatopoulos, “Traffic signal control via reinforcement learning: A review on applications and innovations,” *Infrastructures*, vol. 10, no. 5, p. 114, 2025.
- [6] K. Tang, M. Boltze, H. Nakamura, and Z. Tian, *Global practices on road traffic signal control: Fixed-time control at isolated intersections*. Elsevier, 2019.
- [7] A. Kalašová, A. Hájnik, S. Kubal’ák, J. Beňuš, and V. Harantová, “The impact of actuated control on the environment and the traffic flow,” *Journal of Applied Engineering Science*, vol. 20, no. 2, pp. 305–314, 2022.
- [8] P. Mannion, J. Duggan, and E. Howley, “An experimental review of reinforcement learning algorithms for adaptive traffic signal control,” *Autonomic road transport support systems*, pp. 47–66, 2016.
- [9] B.-L. Ye, W. Wu, K. Ruan, L. Li, T. Chen, H. Gao, and Y. Chen, “A survey of model predictive control methods for traffic signal control,” *IEEE/CAA Journal of Automatica Sinica*, vol. 6, no. 3, pp. 623–640, 2019.
- [10] B.-L. Ye, W. Wu, K. Ruan, L. Li, T. Chen, H. Gao, and Y. Chen, “A survey of model predictive control methods for traffic signal control,” *IEEE/CAA Journal of Automatica Sinica*, vol. 6, no. 3, pp. 623–640, 2019.

- [11] M. Elimadi, A. Abbas-Turki, A. Koukam, M. Dridi, and Y. Mualla, "Review of traffic assignment and future challenges," *Applied Sciences*, vol. 14, no. 2, p. 683, 2024.
- [12] I. Guilliard, S. Sanner, F. W. Trevizan, and B. C. Williams, "Nonhomogeneous time mixed integer linear programming formulation for traffic signal control," *Transportation Research Record*, vol. 2595, no. 1, pp. 128–138, 2016.
- [13] S. Mansouryar, C. Colombaroni, N. Isaenko, and G. Fusco, "Deep reinforcement learning approach for traffic light control and transit priority," *Future Transportation*, vol. 5, no. 4, 2025.

Chapter 7

Appendix A

7.1 PPO Training Script

Listing 7.1: PPO Training Script — Tsotlhe N. Seiphepi (ID: 21001137)

```
"""
Author: _Tsotlhe_Nayang_Seiphepi
Student_ID: _21001137

PPO_Traffic_Signal_Control_Training_Script
This_script_trains_a_Proximal_Policy_Optimization_(PPO)_agent
within_a_SUMO-based_traffic_signal_control_environment.
"""

import os
import time
import torch
from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv
from stable_baselines3.common.callbacks import CheckpointCallback, EvalCallback
from stable_baselines3.common.monitor import Monitor
from env import SumoEnv

def main():
    """
    Main_training_entry_point_for_the_PPO_agent.
    Sets_up_the_environment,_logging_directories,_callbacks_,
    hyperparameters,_and_initiates_training.
    """
```



```

"""

# -----
# ENVIRONMENT SETUP
# -----
sumo_config = "map.sumocfg"

def make_env():
    """Factory_method_to_create_a_monitored_SUMO_environment."""
    return Monitor(SumoEnv(sumo_cfg=sumo_config, use_gui=False))

# Create vectorized environment
env = DummyVecEnv([make_env])

# -----
# LOGGING & DIRECTORY SETUP
# -----
run_name = time.strftime("PPO_run_%Y%m%d_%H%M%S")
log_dir = f"./tb_logs/{run_name}/"
os.makedirs(log_dir, exist_ok=True)

checkpoint_dir = "./checkpoints/"
os.makedirs(checkpoint_dir, exist_ok=True)

# -----
# DEVICE SELECTION
# -----
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"\nUsing_device:_{device}\n")

# -----
# CHECKPOINT CALLBACK
# -----
checkpoint_callback = CheckpointCallback(
    save_freq=5000,
    save_path=checkpoint_dir,
    name_prefix="ppo_traffic"
)

```

```

# -----
# EVALUATION CALLBACK
# -----
eval_env = DummyVecEnv([make_env])
eval_callback = EvalCallback(
    eval_env,
    best_model_save_path="./best_model/",
    log_path="./eval_logs/",
    eval_freq=4000,
    deterministic=True,
    render=False
)

# -----
# PPO MODEL INITIALISATION
# -----
model = PPO(
    policy="MlpPolicy",
    env=env,
    verbose=1,
    tensorboard_log=log_dir,
    device=device,
    learning_rate=3e-4,
    n_steps=1024,
    batch_size=64,
    n_epochs=5,
    gamma=0.99,
    gae_lambda=0.95,
    clip_range=0.2
)

# -----
# TRAINING
# -----
TOTAL_TIMESTEPS = 8000

print(f"      Training PPO Agent for {TOTAL_TIMESTEPS} timesteps ...")
model.learn(
    total_timesteps=TOTAL_TIMESTEPS,

```

```

        callback=[checkpoint_callback, eval_callback],
        tb_log_name="PPO_traffic_signal"
    )

    # -----
    # SAVE FINAL MODEL
    # -----

    model.save("ppo_traffic_final")

    print("\n      _Training_completed_successfully!")
    print("Final_model_saved_as:_ppo_traffic_final.zip")
    print(f"TensorBoard_logs_saved_in:{log_dir}")

if __name__ == "__main__":
    main()

```

```

Warning: Route file should be sorted by departure time, ignoring '4707_to_4659_phase3'!
Warning: Route file should be sorted by departure time, ignoring '4707_to_4659_phase4'!
Warning: Route file should be sorted by departure time, ignoring '4707_to_E6_phase1'!
Warning: Route file should be sorted by departure time, ignoring '4707_to_E6_phase2'!
Warning: Route file should be sorted by departure time, ignoring '4707_to_E6_phase3'!
Warning: Route file should be sorted by departure time, ignoring '4707_to_E6_phase4'!
Warning: Route file should be sorted by departure time, ignoring 'E0_to_E5'!
Warning: Route file should be sorted by departure time, ignoring 'R4659_to_E5'!
Warning: Route file should be sorted by departure time, ignoring '4707_to_E5'!
Warning: Route file should be sorted by departure time, ignoring 'E5_out_to_4707_phase1'!
Warning: Route file should be sorted by departure time, ignoring 'E5_out_to_4707_phase2'!
Warning: Route file should be sorted by departure time, ignoring 'E5_out_to_4707_phase3'!
Warning: Route file should be sorted by departure time, ignoring 'E5_out_to_4707_phase4'!
Warning: Route file should be sorted by departure time, ignoring 'E5_out_to_4659'!
Warning: Route file should be sorted by departure time, ignoring 'E5_out_to_E6'!
-----CI: 0ms, vehicles TOT 9 ACT 9 BUF 0)
| rollout/ |
| ep_len_mean | 488 |
| ep_rev_mean | -30.3 |
| time/ |
| fps | 18 |
| iterations | 4 |
| time_elapsed | 216 |
| total_timesteps | 4096 |
-----
Step #988.00 (0ms ?*RT. ?UPS, TraCI: 5ms, vehicles TOT 102 ACT 0 BUF 0)

```

Figure 7.1: Training the PPO model